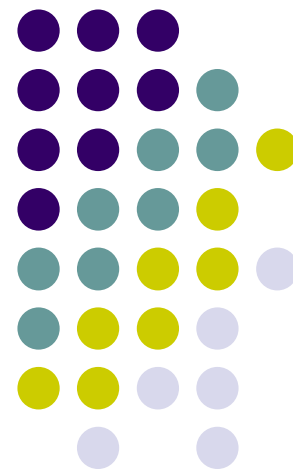
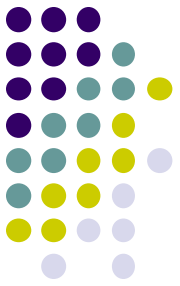


L20: 强连通分量

吉林大学计算机学院
谷方明

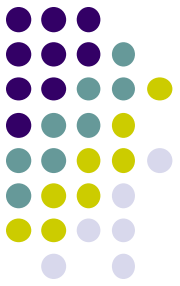
fmgu2002@sina.com





学习目标

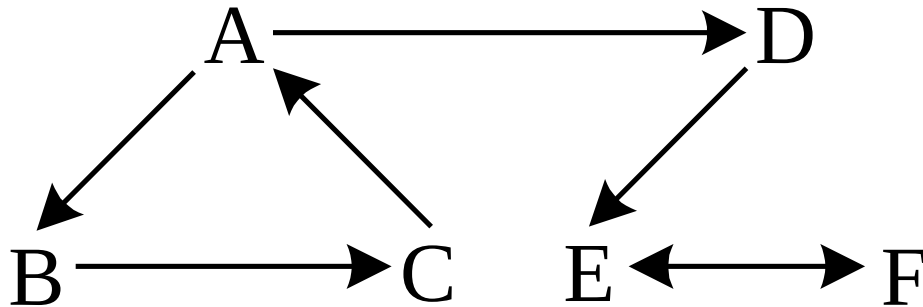
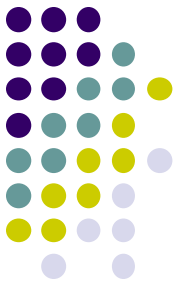
- 理解求强连通分量的**All_Componet**算法
- 掌握求强连通分量的求**Tarjan**算法



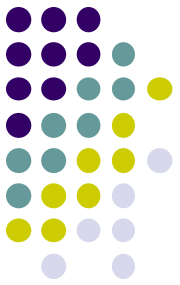
定义回顾

- 有向图中如果两个点能互相可及（或连通）则称这两个点强连通。
- 有向图**G**中任意两点互相可及（或连通），则称**G**是强连通图。
- 有向图**G**的极大强连通子图，称为强连通分量 (SCC, Strongly Connected Component)。

例

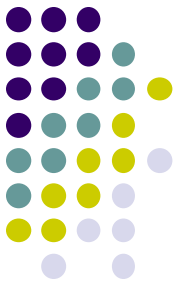


□ 三个强连通分量: $\{A,B,C\}, \{D\}, \{E,F\}$



教材上All_Component

- 思想：根据强连通分量的定义判断两个顶点是否属于同一连通分量；
- 利用Warshall算法求图G的可及矩阵；
- 容器：记录当前SCC中的点
 - ✓ 栈、队列、线性表均可
- `visited[ ]`： `visited[i]`表示结点i是否已处理过



算法All_Component(E.)

*/*输入图的边集E，输出图中所有的强连通分量*/*

All_Component1[初始化]

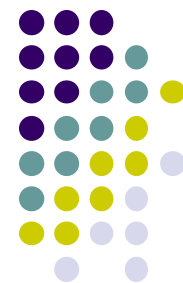
FOR k = 1 TO n DO visited[k] $\leftarrow$ 0.

Warshall(E.WSM). //计算图的可及矩阵

t $\leftarrow$ 0. //记录连通分量的个数

CREATESTACK(scc).

All_Componet2 [计算图的全部连通分量]



```
FOR v=1 TO n DO(
```

```
  IF (visited[v]=0) THEN ( /*处理新的连通分量*/
```

```
    t ← t+1. visited[v] ← 1.
```

```
    FOR i=1 TO n DO(
```

```
      IF( i≠v AND WSM[v][i]=1 AND WSM[i][v]=1) THEN(
```

```
        visited [i] ← 1.
```

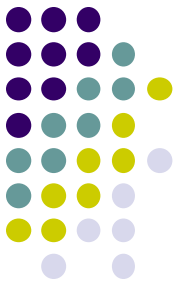
```
        scc.push(i).))
```

```
    PRINT (“第” t “个连通分量: ” ).
```

```
    WHILE(!scc.empty()) DO ( PRINT(scc.pop()).
```

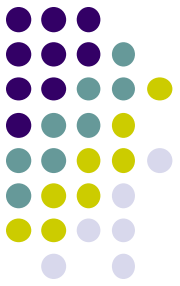
```
  )) █
```

- 算法All_Componet的时间复杂度 $O(n^3)$ ，空间复杂度 $O(n^2)$



Tarjan算法思想

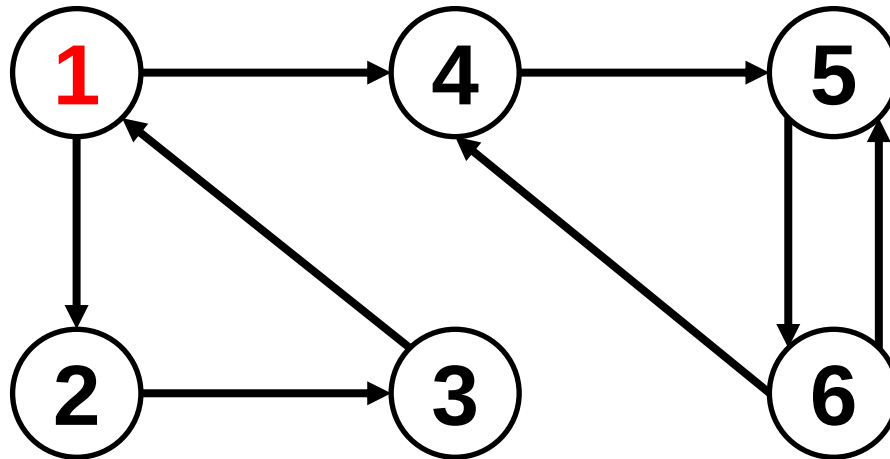
- **SCC性质**：存在一条回路能从初始点经过其中所有点又回到初始点。
- **思想**：处于同一个SCC中的结点必然构成DFS树的一棵子树。找SCC，即找其所在子树的根。
- **辅助结构**
 - ✓ **dfn[u]**：dfs时达到顶点u的次序号（时间戳）；
 - ✓ **low[u]**：u所在DFS子树中次序号最小顶点的次序号；
 - ✓ **Stack**：存储搜索路径上的点；



Tarjan算法示例

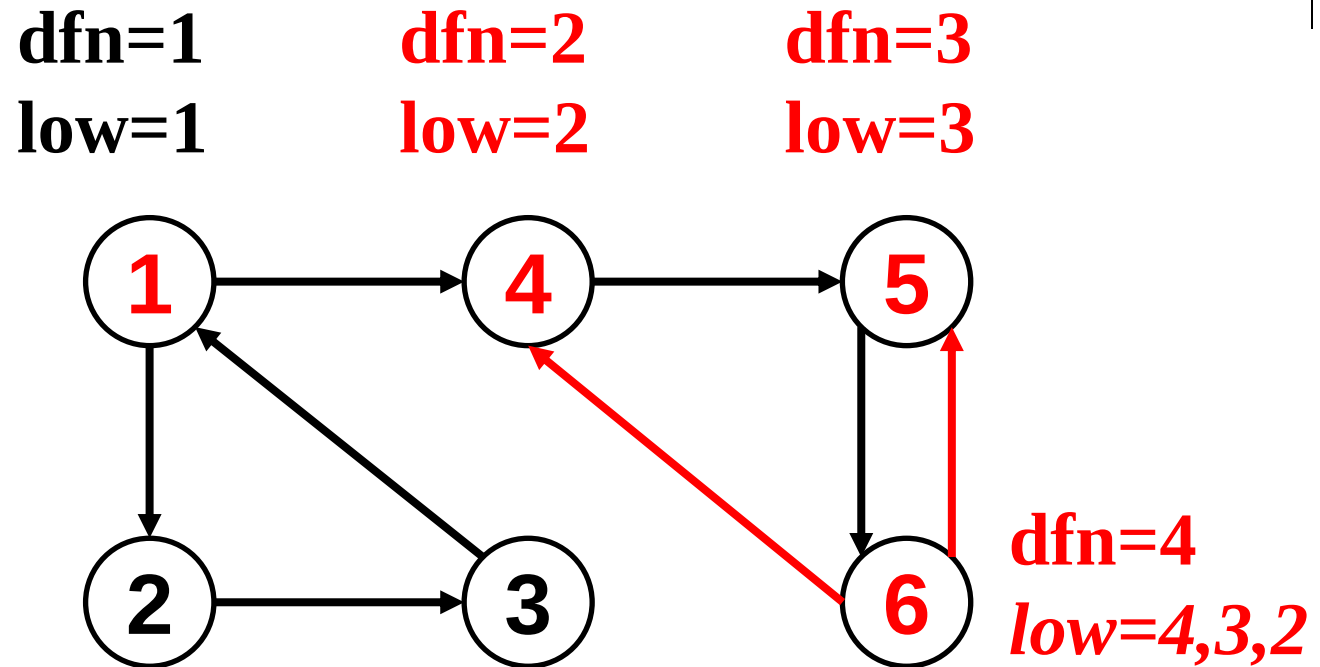
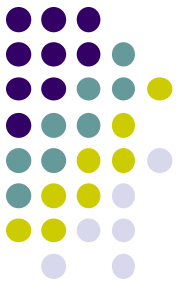
dfn=1

low=1



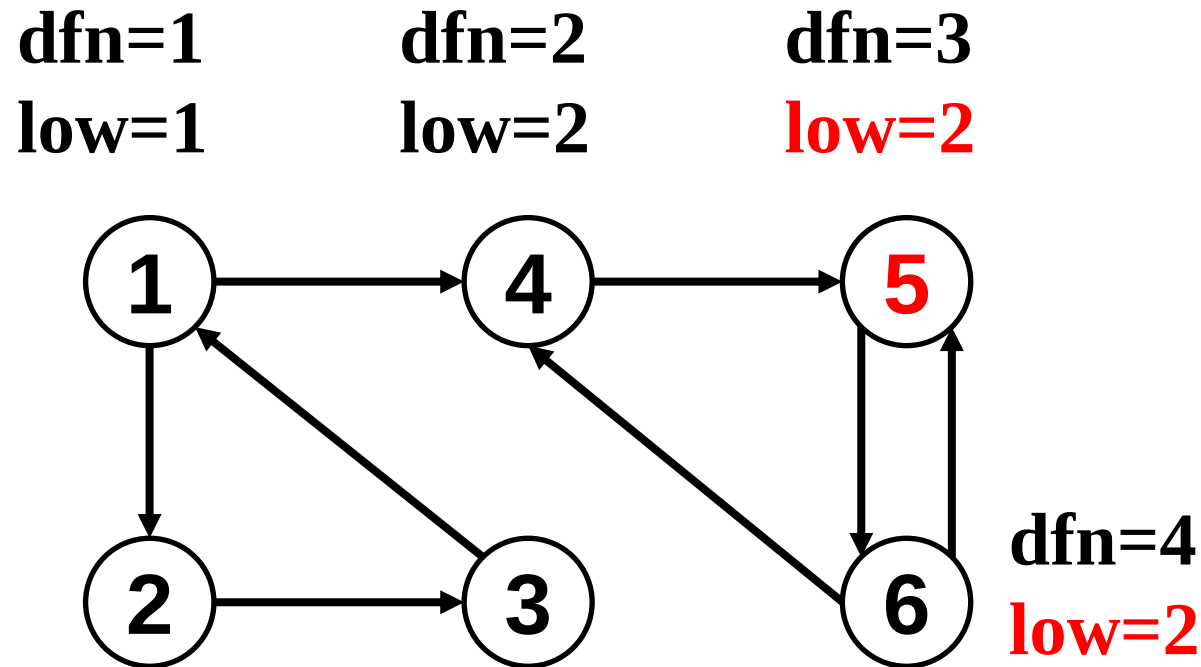
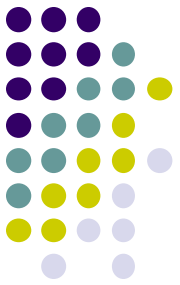
从结点1开始搜索，初始化dfn和low，将结点入栈

Stack: 1



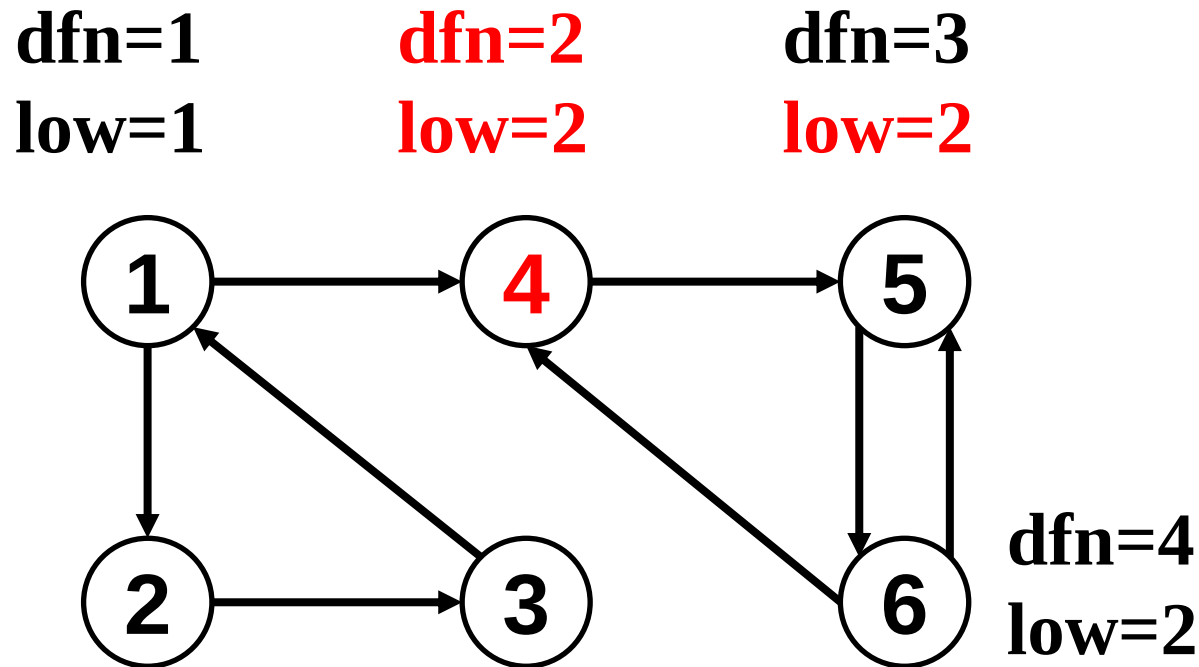
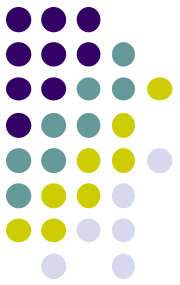
未遇到SCC之前，搜索正常进行，一直到6

Stack: 1, 4, 5, 6



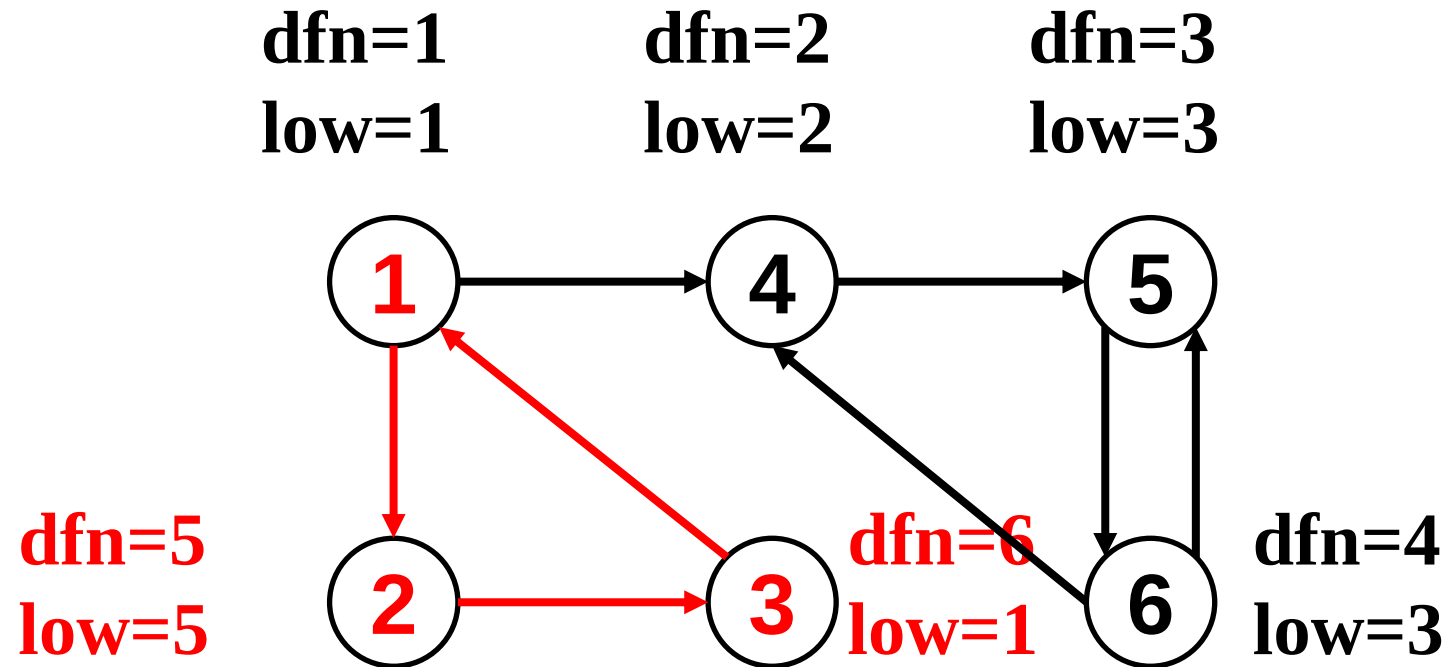
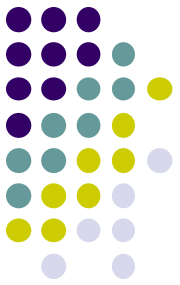
6不能继续搜索；回溯到5，更新low[5]=2

Stack: 1, 4, 5, 6



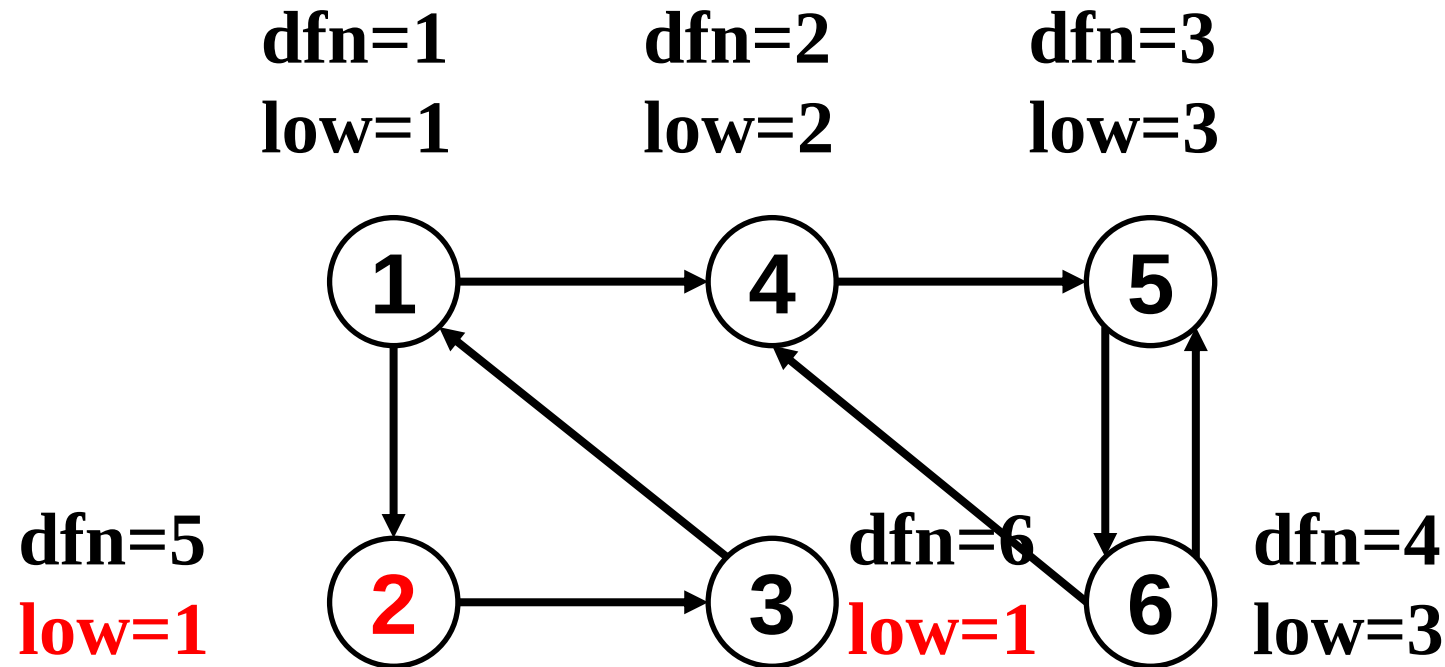
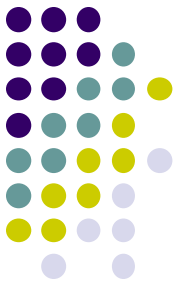
5不能继续搜索；回溯到4， $low[4]$ 不变。4不能继续搜索， $dfn[4]=low[4]$ ，找到一个SCC={4, 5, 6}

Stack: 1



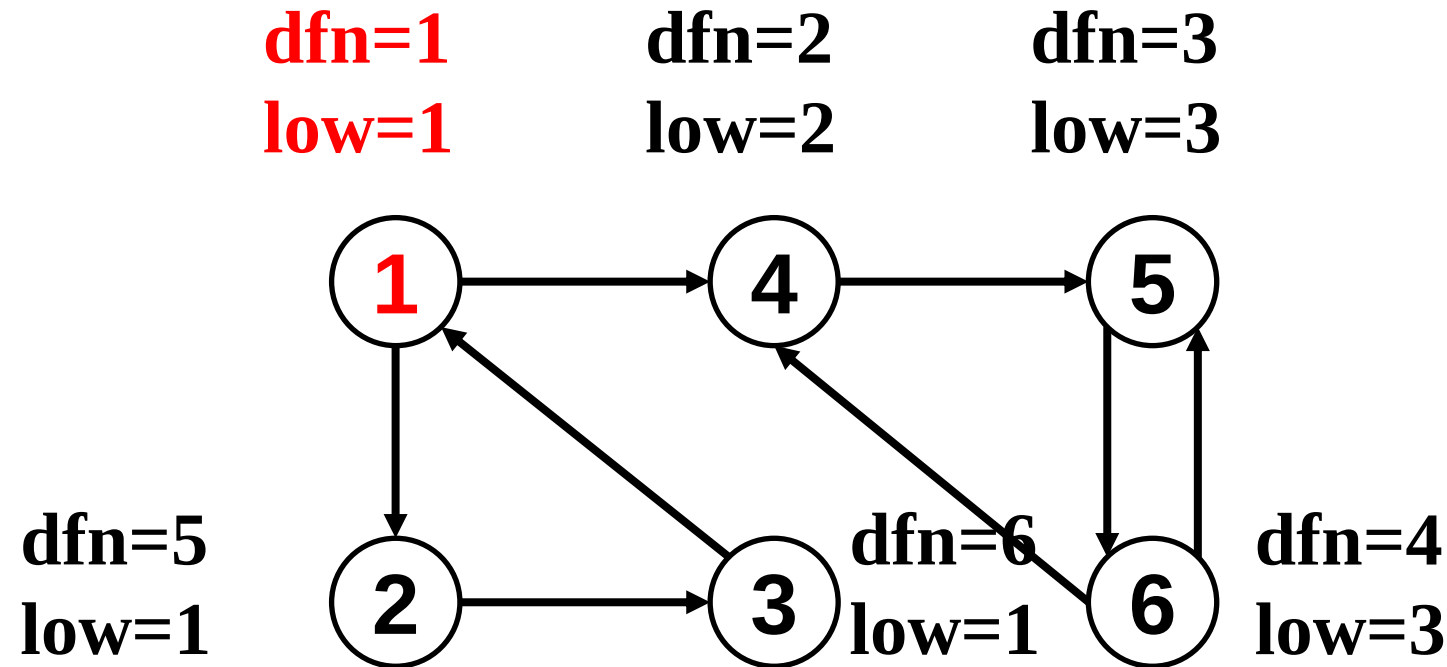
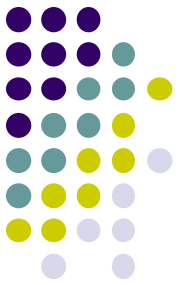
回溯到1，1可以继续搜索，一直到3

Stack: 1, 2, 3



3不能继续搜索；回溯到2，更新low[2]=1

Stack: 1, 2, 3



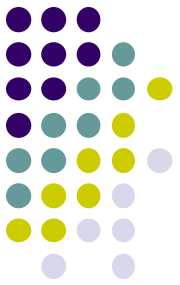
2不能继续搜索；回溯到1，low[1]不变。1不能继续搜索，dfn[1]=low[1]，找到一个SCC={1, 2, 3}

Stack:

Tarjan算法描述

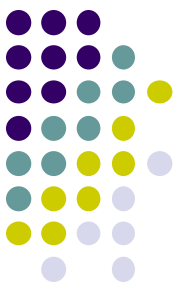


```
tarjan(u){
    DFN[u]=Low[u]=++Index    // 为u设定次序编号和Low初值
    Stack.push(u)            // 将u压入栈中
    for each (u, v) in E     // 枚举每一条边
        if (v is not visted) {    // 如果节点v未被访问过
            tarjan(v)                // 继续向下找
            Low[u] = min(Low[u], Low[v])
        }else if (v in S)           // 如果节点v还在栈内
            Low[u] = min(Low[u], DFN[v])
    if (DFN[u] == Low[u])        // 如果节点u是强连通分量的根
        do{
            v = S.pop // 将v退栈，为该强连通分量中一个顶点
            print v
        }while (u== v)
```

Tarjan算法分析

- 类似DFS，如果求全图的SCC，每次选择一个未被访问的顶点u，做Tarjan算法。
 - ✓ DFN数组可作访问标志
 - ✓ Instack数组：顶点在栈中标志
- Tarjan算法中，每个顶点都被访问了一次，且只进出了一次堆栈，每条边也只被访问了一次，所以该算法的时间复杂度为 $O(n+e)$ 。



Tarjan算法的用途

□ 缩环（成点）

□ 割点

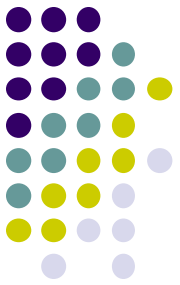
- ✓ 在一个无向图中，如果删除某个顶点，这个图就不再连通

□ 桥（割边）

- ✓ 在一个无向图中删除某条边后，图不再连通

□ 双连通分量

- ✓ 若一个无向图中的去掉任意一个节点（一条边）都不会改变此图的连通性，即不存在割点（桥），则称作点（边）双连通图。



强连通分量的实用算法

□ Kosaraju-Sharir算法

- ✓ 两次DFS，一次原图，一次逆图
- ✓ 参考算法导论/网上教程，直观，但实现略复杂

□ Tarjan算法

- ✓ 利用DFS的dfn和搜索树的根编号low
- ✓ 实测中，Tarjan算法运行效率比Kosaraju算法约高30%

□ Gabow算法

- ✓ Tarjan算法的提升版；利用DFS 和 两个栈；

□ 三个算法都是基于DFS的，都是 $O(n+e)$ 的。